Mathematical Foundations of 3D Trajectory Denoising Using a Fully-Connected Denoising Autoencoder

Thesis Technical Reference | N = 1000, T = 800, σ = 0.6, embedding = 8

1. Data Representation

1.1 Single trajectory

A single 3D spiral trajectory is a multivariate time series — a sequence of T observations, each recording three spatial coordinates at one instant in time. Formally, a trajectory τ is defined as:

$$\tau = \{(x_t, y_t, z_t) \in \mathbb{R}^3 : t = 0, 1, \ldots, T-1\}$$

which can equivalently be written as a matrix:

$$\tau \in \mathbb{R}^{T \times 3} \qquad (\text{rows} = \text{timesteps}, \text{columns} = x, y, z)$$

In this project $T = 800$, meaning each trajectory contains 800 timestep snapshots of the 3D position of a spiralling particle.

1.2 Full dataset

The complete synthetic dataset consists of $N = 1000$ independently generated trajectories, forming a rank-3 tensor:

$$X \in \mathbb{R}^{N \times T \times 3} \quad (1000 \times 800 \times 3)$$

This 3D array contains:

- $N = 1000$ — number of distinct spiral trajectories

- $T = 800$ — timesteps per trajectory
- 3 — spatial coordinates per timestep (x, y, z)

1.3 Flattening for the autoencoder

The fully-connected autoencoder requires a flat 1D vector as input. Each trajectory matrix is therefore unrolled (flattened) into a single row vector:

$$\tau_{\text{flat}} = \text{flatten}(\tau) \in \mathbb{R}^{T \times 3} = \mathbb{R}^{800 \times 3} = \mathbb{R}^{2400}$$

The full dataset then becomes a 2D matrix $X_{\text{flat}} \in \mathbb{R}^{1000 \times 2400}$, where each row is one flattened trajectory. This is what is fed into the autoencoder.

2. Spiral Trajectory Generation Function

2.1 Parametric definition

Each trajectory is generated by sampling a continuous time variable t over the interval $[0, t_{\max}]$ at T evenly-spaced points:

$$t = \text{linspace}(0, t_{\max}, T) \quad \Longrightarrow \quad \Delta t = \frac{t_{\max}}{T - 1}$$

The three spatial coordinates are then defined as:

$$\begin{aligned} r(t) &= r_0 + k \cdot t \qquad \text{(radius at time } t\text{)} \\ x(t) &= r(t)\cos(t) = (r_0 + kt)\cos(t) \\ y(t) &= r(t)\sin(t) = (r_0 + kt)\sin(t) \\ z(t) &= v_z \cdot t \end{aligned}$$

where the four parameters are independently randomised per trajectory:

- $r_0 \sim \text{Uniform}(0.5, 1.5)$ — starting radius

- $k \sim \text{Uniform}(-0.2, 0.2)$ — radius growth rate (positive = expanding, negative = contracting)
- $v_z \sim \text{Uniform}(-0.5, 0.5)$ — vertical speed (positive = rising, negative = falling)
- $t_{\max} \sim \text{Uniform}(2\pi, 6\pi)$ — total arc (1 to 3 full rotations)

2.2 Geometric interpretation

The $x(t)$ and $y(t)$ equations define a spiral in the horizontal plane — circular motion whose radius grows or shrinks linearly over time. The $z(t)$ equation adds independent linear vertical motion, producing a 3D helix whose pitch is controlled by v_z. The combination of randomised parameters across 1000 trajectories creates a rich dataset spanning a wide family of 3D spiral shapes.

2.3 Intrinsic dimensionality

Each trajectory is fully determined by exactly 4 scalar parameters: $r_0, k, v_z, t_{\max}$. This means the intrinsic dimensionality of the trajectory family is 4, even though each trajectory is represented as a vector in $\mathbb{R}^{2400}$. This justifies using a low-dimensional bottleneck — the autoencoder only needs to discover 4 degrees of freedom to perfectly reconstruct any trajectory in the dataset.

3. Noise Model

3.1 Gaussian noise addition

Artificial noise is added independently to every coordinate of every timestep. Given a clean trajectory τ_{clean}, the noisy observation τ_{noisy} is:

$$\tau_{\text{noisy}}(t) = \tau_{\text{clean}}(t) + \varepsilon(t)$$

$$\varepsilon(t) = (\varepsilon_x, \varepsilon_y, \varepsilon_z)(t) \quad \text{where} \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

In this project $\sigma = 0.6$, so each coordinate is perturbed by noise drawn from a zero-mean Gaussian with standard deviation 0.6. The noise is:

- Independent across all timesteps and coordinates
- Independent of the clean signal value
- Identically distributed (i.i.d.) across the full dataset

3.2 Noise power and SNR

The expected noise power (variance) per coordinate per timestep is:

$$E[\varepsilon^2] = \sigma^2 = 0.6^2 = 0.36$$

Signal-to-Noise Ratio in decibels is defined as:

$$\text{SNR (dB)} = 10 \cdot \log_{10}\left(\frac{P_{\text{signal}}}{P_{\text{noise}}}\right) = 10 \cdot \log_{10}\left(\frac{\text{Var}(\tau_{\text{clean}})}{\sigma^2}\right)$$

A higher SNR after reconstruction (compared to before) directly quantifies how much signal the autoencoder recovers. In the results, the AE achieves SNR improvements of +15 to +20 dB, corresponding to a noise power reduction of 32× to 100×.

4. Autoencoder Architecture

4.1 Overview

A denoising autoencoder (DAE) is a neural network trained to map a corrupted input back to its clean version. It consists of two sub-networks — an encoder that compresses the input into a low-dimensional latent representation, and a decoder that reconstructs the clean signal from that representation.

$$f : \mathbb{R}^{2400} \longrightarrow \mathbb{R}^{2400} \qquad \text{(input: noisy trajectory} \rightarrow \text{output: clean trajectory)}$$

4.2 Encoder

The encoder maps the noisy flattened trajectory to a compact latent vector z:

$$\text{Encoder} : \mathbb{R}^{2400} \longrightarrow \mathbb{R}^{128} \longrightarrow \mathbb{R}^{8}$$

It consists of two fully-connected (Linear) layers with ReLU activations:

$$h_1 = \text{ReLU}(W_1 \cdot x_{\text{noisy}} + b_1) \qquad W_1 \in \mathbb{R}^{128 \times 2400},\ b_1 \in \mathbb{R}^{128}$$
$$z = \text{ReLU}(W_2 \cdot h_1 + b_2) \qquad W_2 \in \mathbb{R}^{8 \times 128},\ b_2 \in \mathbb{R}^{8}$$

The latent vector $z \in \mathbb{R}^8$ is the bottleneck — 8 numbers representing the entire 2400-dimensional trajectory. This represents a compression ratio of:

$$\text{Compression ratio} = \frac{2400}{8} = 300\times$$

4.3 Decoder

The decoder maps the latent vector back to a full-length trajectory:

$$\text{Decoder} : \mathbb{R}^8 \longrightarrow \mathbb{R}^{128} \longrightarrow \mathbb{R}^{2400}$$

$$h_3 = \text{ReLU}(W_3 \cdot z + b_3) \qquad W_3 \in \mathbb{R}^{128 \times 8},\ b_3 \in \mathbb{R}^{128}$$
$$\hat{x} = W_4 \cdot h_3 + b_4 \qquad W_4 \in \mathbb{R}^{2400 \times 128},\ b_4 \in \mathbb{R}^{2400}$$

Note: no ReLU is applied after the final Linear layer. This is intentional — trajectory coordinates can be negative (e.g. $x = -1.8$), so constraining the output to non-negative values would prevent accurate reconstruction.

4.4 Full forward pass

The complete forward pass from noisy input to clean reconstruction is:

$$\hat{x}_{\text{clean}} = \text{Decoder}(\text{Encoder}(x_{\text{noisy}}))$$
$$= W_4 \cdot \text{ReLU}\Big(W_3 \cdot \text{ReLU}(W_2 \cdot \text{ReLU}(W_1 x_{\text{noisy}} + b_1) + b_2) + b_3\Big) + b_4$$

4.5 Parameter count

Total trainable parameters across all four Linear layers:

$$W_1, b_1: \ 2400 \times 128 + 128 = 307{,}328 \text{ parameters}$$
$$W_2, b_2: \ 128 \times 8 + 8 = 1{,}032 \text{ parameters}$$
$$W_3, b_3: \ 8 \times 128 + 128 = 1{,}152 \text{ parameters}$$
$$W_4, b_4: \ 128 \times 2400 + 2400 = 309{,}600 \text{ parameters}$$
$$\text{Total} = 307{,}328 + 1{,}032 + 1{,}152 + 309{,}600 = 619{,}112 \text{ parameters}$$

5. Activation Function — ReLU

5.1 Definition

The Rectified Linear Unit (ReLU) is applied element-wise after each hidden Linear layer. It is defined as:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

Applied to a vector, it acts independently on every element:

$$\text{ReLU}([x_1, x_2, \ldots, x_n]) = [\max(0, x_1), \max(0, x_2), \ldots, \max(0, x_n)]$$

5.2 Why ReLU is necessary

Without a non-linear activation function, stacking multiple Linear layers is mathematically equivalent to a single Linear layer. To see why:

$$W_2 \cdot (W_1 \cdot x) = (W_2 W_1) \cdot x = W_{\text{combined}} \cdot x$$

This means two Linear layers without activation collapse into one, and no matter how many layers are stacked, the network can only learn linear transformations. A spiral trajectory is a non-linear function of time — it cannot be represented by any linear mapping. ReLU breaks the linearity and allows the network to approximate arbitrary non-linear functions.

5.3 Derivative and role in backpropagation

The derivative of ReLU used during backpropagation is:

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

This derivative is either 1 (gradient passes through unchanged) or 0 (gradient is blocked). During training, units where the input is positive contribute to learning; units where the input is negative do not update. This sparsity is a feature — it forces different neurons to specialise for different input patterns.

6. Loss Function — Mean Squared Error

6.1 Definition

The network is trained to minimise the Mean Squared Error (MSE) between the reconstructed output and the ground-truth clean trajectory:

$$\mathcal{L}(W) = \frac{1}{N}\sum_{i=1}^{N} \|\hat{x}_i - x_i\|^2 = \frac{1}{N}\sum_{i=1}^{N} \frac{1}{T \cdot 3} \sum_{t=0}^{T-1} \sum_{d \in \{x,y,z\}} (\hat{x}_{i,t,d} - x_{i,t,d})^2$$

where:

- $\hat{x}_i$ = reconstructed (denoised) trajectory for sample i
- x_i = ground-truth clean trajectory for sample i
- N = batch size (64 trajectories per gradient update)
- T = 800 timesteps
- d = spatial dimension (x, y, or z)

6.2 Why MSE for trajectory denoising

MSE penalises large reconstruction errors quadratically — an error of 2 units contributes 4 times more to the loss than an error of 1 unit. This property makes the model prioritise eliminating large noise spikes, which is the desired behaviour for trajectory denoising. MSE also has a well-known closed-form gradient, making it efficient to optimise with gradient descent.

6.3 Relationship to noise variance

A useful baseline is the MSE achieved by a model that always predicts zero (the mean of a zero-centred dataset). This is equal to the signal variance. The noise-only baseline MSE is:

$$\text{MSE}_{\text{baseline}} \approx \text{Var}(x_{\text{clean}}) + \sigma^2 \approx 2.0 + 0.36 \approx 2.08$$

This exactly matches the initial loss of 2.08 seen at epoch 0 in the loss curve, confirming the model started from a mean-prediction state. The final MSE of 0.0122 represents a 170× improvement over this baseline, reducing the error to just 3.4% of the noise variance.

7. Training Procedure

7.1 Optimiser — Adam

Weights are updated using the Adam (Adaptive Moment Estimation) optimiser, which maintains per-parameter learning rates adjusted by gradient history:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \qquad \text{(first moment — gradient mean)}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \qquad \text{(second moment — gradient variance)}$$

$$\theta_t = \theta_{t-1} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \qquad \text{(weight update)}$$

with default values $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, and learning rate $\eta = 0.001$.

7.2 Training loop mathematics

Each training step processes one batch of 64 trajectories:

- Forward pass: $\hat{x} = f(x_{\text{noisy}}; W)$
- Loss: $\mathcal{L} = \text{MSE}(\hat{x}, x_{\text{clean}})$
- Backward pass: $\partial\mathcal{L}/\partial W$ = backpropagation through all layers
- Update: $W \leftarrow W - \text{Adam}(\partial\mathcal{L}/\partial W)$

7.3 Steps and epochs

With $N_{\text{train}} = 600$ and batch_size$= 64$, each epoch consists of:

$$\text{steps per epoch} = \left\lceil \frac{600}{64} \right\rceil = 10 \text{ gradient updates}$$

$$\text{total updates} = 400 \text{ epochs} \times 10 \text{ steps} = 4{,}000 \text{ gradient updates}$$

7.4 Train/test split

The dataset is split before any training. The first 60% of trajectories are used for training and the remaining 40% are held out for evaluation:

$$n_{\text{train}} = \lfloor N \times 0.6 \rfloor = \lfloor 1000 \times 0.6 \rfloor = 600$$

$$n_{\text{test}} = N - n_{\text{train}} = 1000 - 600 = 400$$

The model never sees the 400 test trajectories during training. The final test MSE of 0.0122 is therefore an unbiased estimate of performance on new, unseen data from the same spiral distribution.

8. Why Compression Causes Denoising

The key insight of a denoising autoencoder is that the bottleneck is too small to pass noise through. To understand why, consider what the encoder must do: it must compress 2400 numbers into just 8. The 2400 input numbers contain:

- **Signal:** the smooth spiral structure — low-frequency, highly correlated across timesteps

- **Noise:** i.i.d. Gaussian perturbations — high-frequency, uncorrelated across timesteps

The 8-dimensional bottleneck has enough capacity to encode the smooth spiral (which is determined by only 4 parameters) but not enough to store 2400 independent noise values. The encoder is therefore forced to discard the noise and retain only the correlated underlying structure. The decoder then reconstructs the clean trajectory from this noise-free representation.

This can be stated formally as an information bottleneck:

$$z = \text{Encoder}(x_{\text{noisy}}) \in \mathbb{R}^8$$

$$\text{The bottleneck enforces:} \quad I(z; \text{noise}) \approx 0, \quad I(z; \text{signal}) \gg 0$$

where $I(\cdot;\cdot)$ denotes mutual information. The network learns to maximise the information about the clean signal preserved in z while discarding noise, because the training objective (MSE vs. clean target) directly rewards this.

9. Summary of Mathematical Components

Component	Mathematical Form	Parameter / Value
Trajectory $x(t)$	$x = (r_0 + kt)\cos(t)$	$r_0 \in [0.5, 1.5]$
Trajectory $y(t)$	$y = (r_0 + kt)\sin(t)$	$k \in [-0.2, 0.2]$
Trajectory $z(t)$	$z = v_z \cdot t$	$v_z \in [-0.5, 0.5]$
Noise model	$\varepsilon \sim \mathcal{N}(0, \sigma^2)$	$\sigma = 0.6$
Input dimension	$T \times 3$	$800 \times 3 = 2400$
Encoder layer 1	$h_1 = \text{ReLU}(W_1 x + b_1)$	$W_1 \in \mathbb{R}^{128 \times 2400}$
Encoder layer 2	$z = \text{ReLU}(W_2 h_1 + b_2)$	$W_2 \in \mathbb{R}^{8 \times 128}$
Bottleneck	$z \in \mathbb{R}^8$	Compression: 300×
Decoder layer 1	$h_3 = \text{ReLU}(W_3 z + b_3)$	$W_3 \in \mathbb{R}^{128 \times 8}$
Decoder layer 2	$\hat{x} = W_4 h_3 + b_4$	$W_4 \in \mathbb{R}^{2400 \times 128}$
Activation	$\text{ReLU}(x) = \max(0, x)$	Applied after layers 1, 2, 3

Loss function	$\mathcal{L} = \frac{1}{N}\sum \lVert \hat{x}_i - x_i \rVert^2$	Final test MSE = 0.0122
Optimiser	Adam: $\theta \leftarrow \theta - \eta \hat{m}/(\sqrt{\hat{v}} + \epsilon)$	$\eta = 0.001$
Total parameters	$\sum (W_k \text{ dims} + b_k \text{ dims})$	619,112